SEMAINE 1 · RAPPORT TECHNIQUE

Fondations des LLM

Conception d'une API multi-fournisseurs avec FastAPI, Ollama et Groq

Projet : **Assistant administratif tunisien**, futur chatbot RAG sur le droit du travail, la CNSS et la création d'entreprise

9	2	~10×
étapes réalisées	fournisseurs de LLM	plus de tokens/s (cloud)

Contenu de ce rapport

- Chaque étape de la semaine, dans l'ordre : objectif, théorie, réalisation, résultats réels.
- Les notions clés des LLM : tokens, fenêtre de contexte, température, prompts, sortie structurée, local vs API.
- Les choix d'ingénierie : configuration typée, appels asynchrones, gestion d'erreurs, abstraction des fournisseurs, Docker.
- Les incidents rencontrés (hallucinations, modèle retiré, clé exposée) et leurs leçons.

Auteur Mohamed Amir Essmin
Date 10 octobre 2026
Dépôt github.com/miromiro007/tunisian-admin-assistant

Table des matières

Contexte et objectifs 3

Étape 0 · Environnement et choix des modèles 3

Étape 1 · Structure de projet professionnelle 4

Dépendances . . . 4

Configuration et secrets . . . 4

Logs et point d'entrée FastAPI . . . 5

Étape 2 · Tokens : mesurer le coût d'une langue 5

Étape 3 · Fenêtre de contexte et température 6

Fenêtre de contexte . . . 6

Température et top_p . . . 6

Étape 4 · Prompts, schémas et `POST /chat` 6

Étape 5 · Sortie JSON structurée 7

Étape 6 · Abstraction des fournisseurs : Ollama + Groq 8

Étape 7 · Comparaison des deux modèles 8

Étape 8 · Docker Compose et GitHub 9

Glossaire des termes techniques 10

Préparation aux entretiens 10

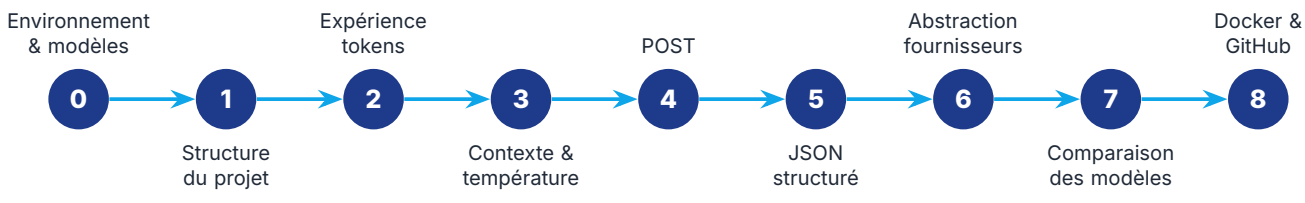
Leçons, checklist et suite 11

Ce que j'ai appris . . . 11

Checklist . . . 12

Suite : semaine 2 . . . 12

La semaine en un coup d'œil



Contexte et objectifs

L'objectif à long terme est un **chatbot RAG** (Retrieval-Augmented Generation, génération augmentée par recherche : le modèle répond à partir de vrais documents et les cite) pour les questions administratives tunisiennes. La semaine 1 s'arrête volontairement avant le RAG et construit les fondations dont tout produit LLM a besoin.

Livrable de la semaine

Une API FastAPI qui dialogue avec un **LLM local** (Ollama, `qwen2.5:7b`) et un **LLM hébergé** (Groq, `openai/gpt-oss-120b`), avec choix du fournisseur, sortie JSON validée par Pydantic, comparaison des modèles, Docker Compose et dépôt GitHub.

Méthode. Chaque étape a suivi le même schéma : objectif, théorie (définition simple, analogie, version technique), code complet, commandes, résultat attendu, erreurs fréquentes, quiz, question d'entretien et commit Git.

ÉTAPE 0

Environnement et choix des modèles

0

Objectif : dimensionner le modèle local selon le matériel, puis choisir une API cloud gratuite.

Composant	Valeur	Impact
CPU	Intel i5-12500H	suffisant pour l'API, lent pour l'inférence CPU
RAM	16 Go	suffisant, Docker Desktop peut en prendre une grande part
GPU	RTX 4060 Laptop, **8 Go de VRAM** (75 W)	composant clé pour les LLM locaux
Logiciels	Windows, Python 3.13.5, Docker 28.4, Compose 2.39	tout est prêt

Théorie Pour répondre, un modèle garde tous ses poids en mémoire rapide (la VRAM). Mémoire ≈ paramètres × octets par poids. En 16 bits, un modèle 7B demande ∼14 Go. La **quantization** (quantification) stocke chaque poids sur moins de bits : en **Q4** (≈ 4 bits), il descend à ∼4,7 Go. Il faut ajouter de la place pour le **KV cache** (mémoire de travail de la conversation) et ∼1 Go utilisé par Windows.

Analogie La VRAM est un plan de travail de cuisine. Si tous les ingrédients y tiennent, le chef va vite. Sinon, il fait des allers-retours vers la réserve (la RAM) et la cuisine devient 5 à 10 fois plus lente.

Taille	Mémoire Q4	Tient dans 7 Go ?	Verdict
3B	∼2,5 Go	oui	très rapide, moins bon
7–8B	∼4,7–5 Go	**oui**	**le bon compromis**
12–14B	∼8–9 Go	non	déborde en RAM, 5 à 10× plus lent

Choix. Local : `qwen2.5:7b` (multilingue EN/FR/AR, fiable en JSON). Cloud : **Groq** (offre gratuite, sans carte bancaire, API compatible OpenAI). Garder les deux est exigé par le programme et relève de la bonne ingénierie : confidentialité et hors-ligne en local, qualité et vitesse dans le cloud.

Résultat `ollama ps` → `qwen2.5:7b 4.7 GB 100% GPU context 4096`. `nvidia-smi` → 4631 / 8188 Mio utilisés. Ollama est un serveur HTTP local sur `http://localhost:11434`.

ÉTAPE 1

Structure de projet professionnelle

1

Objectif : créer une base FastAPI propre avec configuration typée, logs, protection des secrets et route de santé.

```
tunisian-admin-assistant/
|-- app/
|   |-- __init__.py, config.py, logging_config.py, main.py, schemas.py
|   `-- llm/  base.py, ollama_provider.py, groq_provider.py,
|             factory.py, prompts.py, errors.py
|-- scripts/  count_tokens.py, temperature_test.py, compare_models.py
|-- .env (ignore)   .env.example   .gitignore   requirements.txt
`-- Dockerfile   docker-compose.yml   .dockerignore   README.md
```

Dépendances

Paquet	Rôle dans le projet	Équivalent .NET
fastapi	routes, validation, documentation Swagger automatique	ASP.NET Core Web API
uvicorn	serveur ASGI qui écoute les requêtes HTTP	Kestrel
pydantic	valide requêtes, réponses et sorties du LLM	DTO + DataAnnotations
pydantic-settings	configuration typée lue depuis `.env`	`IOptions<T>`
httpx	client HTTP asynchrone vers Ollama et Groq	`HttpClient`
tiktoken	compte les tokens (approximation)	–

Configuration et secrets

Théorie La configuration est séparée du code (principe **12-factor**). `.env` contient les vraies valeurs et est ignoré par Git ; `.env.example` contient des valeurs fictives et est versionné. `BaseSettings` lit chaque variable, la convertit et valide son type au démarrage (**fail fast**, échouer tôt). `SecretStr` affiche `**********` ; la vraie clé n'est lue qu'avec `.get_secret_value()`, là où elle est envoyée. `@lru_cache` construit la configuration une seule fois (singleton). Priorité : variable d'environnement réelle > `.env` > valeur par défaut.

```python
class Settings(BaseSettings):
    ollama_base_url: str = "http://localhost:11434"
    ollama_model: str = "qwen2.5:7b"
    groq_api_key: SecretStr = SecretStr("")   # masquee dans les logs
    groq_model: str = "openai/gpt-oss-120b"
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Attention Remplir `.gitignore` **avant** de créer `.env`, et vérifier avec `git check-ignore .env`. Une faute de frappe comme `GROQ_API_KEYY` ne provoque aucun crash : `extra="ignore"` l'ignore et la clé reste vide.

Logs et point d'entrée FastAPI

Les logs sortent sur **stdout** au format `heure | niveau | module | message`, pour que Docker les collecte. Niveaux : `DEBUG < INFO < WARNING < ERROR < CRITICAL` ; la production utilise INFO, et les secrets ne sont jamais journalisés. `main.py` crée l'application, un hook `lifespan` (le code avant `yield` s'exécute au démarrage, après lui à l'arrêt) et `GET /health`, une sonde de vie pour Docker et les répartiteurs de charge.

Résultat `Starting Tunisian Admin Assistant (env=dev),  GET /health 200 OK  →  {"status":"ok","environment":"dev"}`, interface Swagger sur `/docs`.

Lien .NET `main.py` ≈ `Program.cs`, `@app.get` ≈ `[HttpGet]`, `lifespan` ≈ `IHostedService`, `.venv` ≈ un jeu de paquets propre à la solution.

ÉTAPE 2

Tokens : mesurer le coût d'une langue

Objectif : voir comment la même phrase est découpée en anglais, français et arabe.

2

Théorie Un LLM lit des **tokens**, pas des mots. Un **tokenizer** (souvent **BPE**, Byte Pair Encoding) construit son vocabulaire en fusionnant les paires d'octets les plus fréquentes de ses données d'entraînement. Chaque token devient un identifiant entier. Le coût, la vitesse et la fenêtre de contexte se comptent en tokens.

Analogie Des briques LEGO : un mot courant est une grosse brique, un mot rare est construit avec plusieurs petites.

Langue	Caractères	Tokens	Car./token
Anglais	39	9	4,33
Français	41	8	5,12
Arabe	30	**25**	**1,20**

Découpage observé : l'anglais garde les mots entiers (`' register'`, `' company'`) ; le français coupe `' Tunis' + 'ie'` ; l'arabe est découpé presque lettre par lettre, et le dernier caractère en deux octets bruts (affichés `\ufffd`). Seuls les mots très courants comme « dans » restent entiers.

Pourquoi c'est important L'arabe coûte ∼2,8× plus de tokens que l'anglais pour le même sens, car les données d'entraînement du tokenizer sont dominées par l'anglais. Pour un assistant tunisien, cela signifie un coût plus élevé et une fenêtre de contexte qui se remplit plus vite, ce qui guidera le découpage des documents en RAG. `tiktoken` est le tokenizer d'OpenAI : pour Qwen et Llama, ce n'est qu'une approximation, les comptes exacts viennent des réponses des API.

ÉTAPE 3

Fenêtre de contexte et température

3

Objectif : comprendre la limite de mémoire du modèle et son curseur d'aléatoire, puis les tester.

Fenêtre de contexte

Nombre maximal de tokens visibles à la fois : prompt + historique + réponse. Au-delà, l'ancien contenu est **tronqué** (oublié) ou la requête échoue. Un LLM est **stateless** (sans état) : il ne retient rien entre deux appels, donc un chatbot doit renvoyer tout l'historique à chaque fois. Ollama utilise une fenêtre par défaut de 4096 tokens.

Température et top_p

À chaque étape, le modèle attribue une probabilité à chaque token possible. La **température** remodèle ces probabilités : basse, le choix le plus probable domine ; haute, elles s'aplatissent. **top_p** ne garde que les tokens les plus probables dont la somme atteint p. On modifie l'un ou l'autre, pas les deux.

Tâche	Température
Extraction, classification, faits juridiques/administratifs	**0 – 0,2**
Questions-réponses générales	0,3 – 0,7
Écriture créative, brainstorming	0,8 – 1,2

Résultat Prompt « Give me a funny name for a Tunisian cafe », 3 essais par réglage, via `httpx` :
$T = 0$: *Carrot Cafe Caravan* trois fois, identique (in=40, out=43).
$T = 1{,}2$: *Carrot Café Caravan*, *Desert Dreams Café*, *Carpet Café Coffee* (out = 38, 26, 35), l'une dérivant même vers « marocain ».
Un prompt de 10 mots a coûté **40 tokens d'entrée** : Ollama l'enveloppe dans le modèle de chat de Qwen (marqueurs de rôle cachés).

Attention Dans PowerShell, `curl` est un alias de `Invoke-WebRequest`. Utiliser `cmd`, `Invoke-RestMethod`, ou plus simplement un script Python avec `httpx`.

ÉTAPE 4

Prompts, schémas et `POST /chat`

4

Objectif : envoyer une question au LLM via notre propre API, avec entrée et sortie validées.

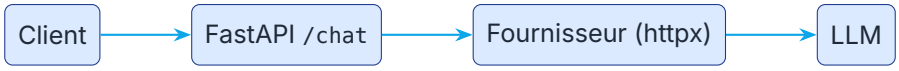
Théorie Les API de chat reçoivent une liste de messages avec des **rôles** : **system** (règles du développeur : rôle, langue, « n'invente jamais de lois » ; priorité maximale, invisible pour l'utilisateur), **user** (la question), **assistant** (réponses précédentes, renvoyées comme historique). Les exemples **few-shot** améliorent la cohérence du format.

Analogie Un restaurant : le message système est le briefing du gérant, le message utilisateur est la commande du client.

```
class ChatRequest(BaseModel):
```

```
    question: str = Field(..., min_length=1, max_length=2000)
    provider: Literal["ollama", "groq"] = "ollama"
    temperature: float = Field(default=0.2, ge=0.0, le=2.0)
```

Une question vide est rejetée en **HTTP 422** avant toute dépense de tokens ; `max_length` protège contre les prompts énormes et coûteux. La route appelle le fournisseur avec `httpx.AsyncClient` : l'**asynchrone** permet au serveur de continuer à servir les autres utilisateurs pendant l'attente. Les échecs sont convertis en codes clairs : connexion refusée → **503**, délai dépassé → **504**, erreur du fournisseur → **502**.



Résultat « How do I create a company in Tunisia ? » → 200 OK, `prompt_tokens`=69, `completion_tokens`=253, `latency_ms`=18366 (le premier appel inclut le chargement du modèle en VRAM).

Attention **Hallucination.** La réponse semblait sûre d'elle mais était en partie fausse (ouvrir un compte bancaire « avec votre numéro CNSS », inscription CNSS avant que la société existe). Un modèle 7B connaît mal l'administration tunisienne : c'est l'argument principal pour un RAG avec sources citées.

ÉTAPE 5

Sortie JSON structurée

5

Objectif : transformer une question en données validées : `theme`, `intent`, `keywords`.

Théorie Un texte libre ne peut pas alimenter d'autre code ; du JSON, si. Trois couches de fiabilité : (1) un prompt clair avec exemples **few-shot** à température 0 ; (2) le **décodage contraint**, le paramètre `format` d'Ollama reçoit un JSON Schema généré par Pydantic ; (3) la **validation Pydantic** avec `model_validate_json`. On ne fait jamais confiance à un système externe (**défense en profondeur**).

```
class QuestionAnalysis(BaseModel):
    theme: Literal["labor_law", "cnss", "company_creation", "taxes", "other"]
    intent: str = Field(..., min_length=1, max_length=200)
    keywords: list[str] = Field(..., min_length=1, max_length=8)
```

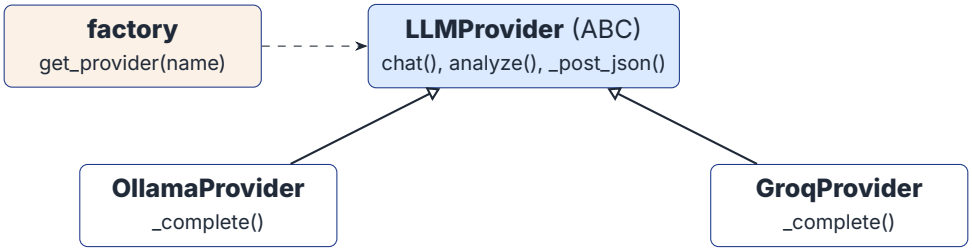
`Literal` fonctionne comme un `enum` : une réponse comme `"theme":"legal"` lève une `ValidationError`, convertie en erreur 502 propre, et n'atteint jamais le reste du système.

Résultat « Quels documents faut-il pour créer une SARL en Tunisie ? » → `theme: company_creation`, intention « Identify required documents for creating a SARL in Tunisia », mots-clés en anglais alors que la question était en français : les petits modèles ne suivent les consignes qu'en partie.

ÉTAPE 6
Abstraction des fournisseurs : Ollama + Groq
6

Objectif : gérer deux fournisseurs interchangeables sans dupliquer le code.

Théorie Une classe abstraite `LLMProvider` définit la « forme de prise » commune. Avec le patron **template method**, `base.py` implémente `chat()` et `analyze()` une seule fois (chronométrage, logs, conversion des erreurs, validation) et chaque fournisseur n'implémente que `_complete()`, la partie HTTP qui diffère. Une **factory** `get_provider(name)` renvoie le bon objet. Ajouter Gemini demande une nouvelle classe, une ligne dans la factory et une valeur dans le `Literal` : c'est le **principe ouvert/fermé**.



	Ollama	Groq (compatible OpenAI)
URL	`{base}/api/chat`	`.../openai/v1/chat/completions`
Authentification	aucune	`Authorization: Bearer <clé>`
Réponse	`message.content`	`choices[0].message.content`
Tokens	`prompt_eval_count`, `eval_count`	`usage.prompt_tokens`, `completion_tokens`
JSON	`format` = JSON Schema complet	`response_format: json_object` (JSON valide, pas notre schéma)

Attention **Modèle retiré.** `llama-3.3-70b-versatile` a renvoyé **404 model_not_found**. La liste `/openai/v1/models` interrogée avec notre clé a montré les modèles disponibles ; nous sommes passés à `openai/gpt-oss-120b` en modifiant une seule ligne de `.env`. Les noms de modèles relèvent de la configuration, pas du code.

Résultat Groq `/chat` : 129 tokens de prompt / 831 de réponse, **2870 ms**. Groq `/analyze` : mots-clés en français (`créer`, `Tunisie`), **1322 ms**. Réponses plus riches, mais détails juridiques toujours non vérifiés.

ÉTAPE 7
Comparaison des deux modèles
7

Objectif : mesurer vitesse, tokens et qualité sur les mêmes 8 questions.

Théorie Une **évaluation** équitable : mêmes prompts, même température (0), une requête de **warm-up** (échauffement) au préalable pour exclure le chargement du modèle en VRAM, et les **tokens par seconde** plutôt que la latence totale (qui dépend de la longueur de la réponse). Questions mêlant EN/FR/AR plus une question hors-sujet de contrôle. La qualité exige un jugement humain face aux sources officielles.

Modèle	Latence moy.	Tokens/s	Tokens prompt	Tokens réponse
`qwen2.5:7b` (local)	4203 ms	25,0	71	128
`gpt-oss-120b` (Groq)	1828 ms	**240,9**	130	490



Pourquoi c'est important Groq génère ∼10× plus de tokens par seconde, mais sa latence totale n'est que ∼2,3× plus faible car ses réponses sont ∼4× plus longues, en partie à cause de tokens de raisonnement cachés mais facturés. Les deux tokenizers comptent différemment le même prompt (71 contre 130). Huit questions donnent une indication, pas un benchmark scientifique.

	Local (Ollama)	API (Groq)
Confidentialité	données sur la machine	données envoyées à un tiers
Coût	gratuit, illimité	quota gratuit, limites (HTTP 429)
Hors-ligne	oui	non
Qualité / vitesse	7B, dépend du GPU, ∼25 tok/s	∼120B, ∼240 tok/s

ÉTAPE 8

Docker Compose et GitHub

8

Objectif : lancer API + Ollama en une commande, puis publier le code en sécurité.

Théorie **Image** = le paquet, **conteneur** = une instance en cours d'exécution. Le **Dockerfile** construit l'API (Python 3.12-slim, requirements copiés d'abord pour le cache des couches, utilisateur non-root, `--host 0.0.0.0`). **Compose** lance `api` et `ollama` sur un même réseau où **les noms de service sont des noms d'hôte** : l'API appelle `http://ollama:11434`, car dans un conteneur `localhost` désigne le conteneur lui-même. Un **volume** conserve les 4,7 Go du modèle. Les secrets sont injectés à l'exécution avec `env_file`, et `.dockerignore` exclut `.env` de l'image. Un **healthcheck** appelle `/health` ; le GPU est transmis via une réservation de périphérique NVIDIA.

Analogie Une aire de restauration : chaque stand (conteneur) a sa propre cuisine, et tous partagent un bâtiment avec une ligne téléphonique interne (le réseau Compose).

Attention **Secret dans l'historique Git.** Une vraie clé Groq s'est retrouvée dans `.env.example`. La **push protection** de GitHub a bloqué l'envoi (`GH013`). Supprimer le fichier dans un nouveau commit n'a pas suffi, car un push envoie tout l'historique. Solution : révoquer la clé, remettre des valeurs fictives, recréer l'historique, vérifier avec `git grep "gsk_"`, puis pousser. Ne jamais cliquer sur « allow the secret ».

Résultat Push réussi : `main -> main`. Les fichiers Docker Compose sont écrits ; le lancement de la pile (`docker compose up -d --build`) reste à vérifier.

Glossaire des termes techniques

Terme	Signification
LLM	Large Language Model, grand modèle de langage qui prédit le token suivant
Token / tokenizer / BPE	unité de texte lue par le modèle / outil de découpage / algorithme Byte Pair Encoding
Fenêtre de contexte	nombre maximal de tokens visibles (prompt + historique + réponse)
Stateless	le modèle ne garde rien entre deux requêtes
Température / top_p	aléatoire de l'échantillonnage / seuil d'échantillonnage par noyau
System / user / assistant	rôles des messages : règles du développeur / question / réponses précédentes
Few-shot	exemples dans le prompt pour montrer le format attendu
Hallucination	contenu plausible mais faux produit par le modèle
Sortie structurée	réponse imposée dans un schéma fixe (JSON)
Décodage contraint	génération limitée à un JSON Schema
Quantization (Q4)	stocker les poids sur moins de bits pour économiser la mémoire
VRAM / KV cache	mémoire du GPU / mémoire de travail de la conversation
Latence / tokens par seconde	temps de réponse total / vitesse de génération
RAG	Retrieval-Augmented Generation, réponse à partir de documents retrouvés
ASGI / uvicorn	interface de serveur web Python asynchrone / son serveur
DTO	Data Transfer Object, classe de requête ou de réponse
SecretStr	type Pydantic qui masque les secrets à l'affichage
Fail fast	échouer dès le démarrage si la configuration est invalide
ABC / template method / factory	classe abstraite / algorithme commun à une étape variable / créateur d'objets
Principe ouvert/fermé	ouvert à l'extension, fermé à la modification
Healthcheck / sonde de vie	route utilisée par les plateformes pour savoir si l'app tourne
Volume	stockage persistant hors du conteneur
Push protection	fonction GitHub qui bloque les envois contenant des secrets

Préparation aux entretiens

Question d'entretien

Q. *Pourquoi l'arabe coûte-t-il plus de tokens que l'anglais ?*

A. Le vocabulaire du tokenizer est appris surtout sur des données dominées par l'anglais, donc les mots arabes sont découpés en nombreux petits morceaux, parfois en octets bruts. Dans notre test, la même phrase a coûté 25 tokens en arabe contre 9 en anglais, ce qui pèse sur le coût et sur le contexte.

Question d'entretien

Q. *Pourquoi mettre la température à 0 pour certaines tâches ?*

A. La température contrôle l'aléatoire de l'échantillonnage. Pour l'extraction, la classification

ou des réponses factuelles, je veux le token le plus probable à chaque fois : les sorties sont stables, testables et dérivent moins.

Question d'entretien

Q. *Comment obtenir une sortie structurée fiable d'un LLM ?*

A. Un prompt few-shot à basse température, un décodage contraint par JSON Schema quand le fournisseur le permet, puis une validation Pydantic, car le mode JSON garantit un JSON valide mais pas mon schéma. En cas d'échec, je journalise la sortie brute et je renvoie une erreur claire.

Question d'entretien

Q. *Comment gérer plusieurs fournisseurs de LLM ?*

A. Une interface abstraite, une classe par fournisseur, la logique commune dans la classe de base, et une factory. Les routes dépendent de l'abstraction : ajouter un fournisseur, c'est une classe de plus. Cela évite la dépendance à un fournisseur et facilite comparaison et bascule.

Question d'entretien

Q. *Comment gérer les secrets, en local et dans Docker ?*

A. Configuration séparée du code : `.env` ignoré par Git, `.env.example` avec des valeurs fictives, `SecretStr` pour que les clés n'atteignent jamais les logs. Dans Docker, jamais de secret dans l'image : `.dockerignore` et injection à l'exécution. Si une clé fuit, je la révoque d'abord, puis je nettoie l'historique.

Question d'entretien

Q. *Comment choisir entre un modèle local et une API hébergée ?*

A. J'évalue les deux sur le même jeu de tâches à température fixe : latence, tokens/s, coût, qualité. Puis je pèse confidentialité, besoin hors-ligne, limites de débit et matériel. Un hybride qui route selon la sensibilité ou la difficulté est souvent le meilleur choix.

Leçons, checklist et suite

Ce que j'ai appris

- Les LLM lisent des tokens ; l'arabe coûte ∼2,8× plus cher que l'anglais dans notre test.
- La fenêtre de contexte est partagée par le prompt et la réponse ; l'historique doit être renvoyé car les LLM sont sans état.
- Température 0 : sortie reproductible ; 1,2 : variété et dérive.
- Le prompt système fixe les règles ; les exemples few-shot fixent le format.
- Ne jamais faire confiance à la sortie d'un LLM : schéma + validation Pydantic.
- Local (privé, gratuit, ∼25 tok/s) contre cloud (∼240 tok/s, plus riche, quotas).
- Les noms de modèles vont dans la configuration : les fournisseurs retirent des modèles.
- Les deux modèles hallucinent des détails juridiques : le RAG avec sources est indispensable.
- Un secret dans l'historique Git y reste : révoquer d'abord, réécrire ensuite.

Checklist

Élément	Statut
`.env` ignoré, `.env.example` avec valeurs fictives	**fait**
Configuration typée avec `SecretStr`, logs sur stdout	**fait**
`GET /health`, `POST /chat`, `POST /analyze`	**fait**
Fournisseurs Ollama et Groq, factory	**fait**
Comparaison sur 8 questions	**fait**
Dépôt poussé sur GitHub	**fait**
Pile Docker Compose vérifiée en fonctionnement	**à vérifier**
Clé Groq exposée révoquée	**à confirmer**
Notes de qualité humaines (1 à 5) par réponse	**à ajouter**

Suite : semaine 2

Introduire le RAG : **embeddings** (texte transformé en vecteurs), une **base vectorielle**, le **chunking** (découpage des documents, en tenant compte du coût des tokens en arabe) et les **citations de sources**, pour que l'assistant réponde à partir des textes officiels tunisiens plutôt que de sa mémoire.